

WordCount.java

```
import java.io.IOException;
import java.util.*;
import org.apache.hadoop.conf.*;
import org.apache.hadoop.fs.*;
import org.apache.hadoop.io.*;
import org.apache.hadoop.mapreduce.*;
import org.apache.hadoop.mapreduce.lib.input.*;
import org.apache.hadoop.mapreduce.lib.output.*;
import org.apache.hadoop.util.*;

public class WordCount extends Configured implements Tool
{
    public static void main(String args[]) throws Exception {
        int res = ToolRunner.run(new WordCount(), args);
        System.exit(res);
    }

    public int run(String[] args) throws Exception {
        Path inputPath = new Path(args[0]);
        Path outputPath = new Path(args[1]);

        Configuration conf = getConf();
        Job job = new Job(conf, this.getClass().toString());
        job.setJarByClass(WordCount.class);

        FileInputFormat.setInputPaths(job, inputPath);
        FileOutputFormat.setOutputPath(job, outputPath);

        job.setJobName("WordCount");

        job.setMapperClass(Map.class);
        job.setCombinerClass(Reduce.class);
        job.setReducerClass(Reduce.class);
        job.setMapOutputKeyClass(Text.class);
        job.setMapOutputValueClass(IntWritable.class);
        job.setOutputKeyClass(Text.class);
        job.setOutputValueClass(IntWritable.class);
        job.setInputFormatClass(TextInputFormat.class);
        job.setOutputFormatClass(TextOutputFormat.class);
        return job.waitForCompletion(true) ? 0 : 1;
    }
}
```

```

    public static class Map extends Mapper<LongWritable, Text, Text,
IntWritable>
    {
        private final static IntWritable one = new IntWritable(1);
        private Text word = new Text();
        @Override
        public void map(LongWritable key, Text value, Mapper.Context
context) throws IOException, InterruptedException
        {
            String line = value.toString();
            StringTokenizer tokenizer = new StringTokenizer(line);
            while (tokenizer.hasMoreTokens()) {
                word.set(tokenizer.nextToken());
                context.write(word, one);
            }
        }
    }
}

```

```

    public static class Reduce extends Reducer<Text, IntWritable, Text,
IntWritable>
    {
        @Override
        public void reduce(Text key, Iterable<IntWritable> values,
Context context) throws IOException, InterruptedException
        {
            int sum = 0;
            for(IntWritable value : values) sum += value.get();
            context.write(key, new IntWritable(sum));
        }
    }
}

```

FindLargest.scala

```
object ExFindLargest {
  def main(args: Array[String]) {
    var number1=20;
    var number2=30;
    var x = 10;
    if( number1>number2){
      println("Largest number is:" + number1);
    }
    else{
      println("Largest number is:" + number2);
    }
  }
}
└─(vmsin0@martian)-[~/STUDY/Manual/DSBDA/EXP12]
└─$ scalac FindLargest.scala && scala ExFindLargest
Largest number is:30
```

Greet.scala

```
object ExPrintName {
  def main(args: Array[String]) {
    println("Hello Mike!")
  }
}
└─(vmsin0@martian)-[~/STUDY/Manual/DSBDA/EXP12]
└─$ scalac Greet.scala && scala ExPrintName
My name is Mike!
```

Number.scala

```
object ExCheckNumber {
  def main(args: Array[String]) {
    var number= (-100);
    if(number==0)
      println("number is zero");
    else if(number>0)
      println("number is positive");
    else
      println("number is negative");
  }
}
└─(vmsin0@martian)-[~/STUDY/Manual/DSBDA/EXP12]
└─$ scalac Number.scala && scala ExCheckNumber
number is negative
```

Calculator.scala

```
object Calculator {
  def main(args: Array[String]) {
    var ch:Char=0
    var num1:Int=0
    var num2:Int=0
    var result:Int=0
    print("Choose operation to perform (+,-,*,/,%): ")
    ch=scala.io.StdIn.readChar()
    print("Enter first number: ")
    num1=scala.io.StdIn.readInt()
    print("Enter second number: ")
    num2=scala.io.StdIn.readInt()
    ch match{
      case '+'=> result=num1+num2
      case '-'=> result=num1-num2
      case '*'=> result=num1*num2
      case '/'=> result=num1/num2
      case '%'=> result=num1%num2
      case _=>printf("Invalid operation.\n")
    }
    println("Result: "+result);
  }
}

(vmsin0@martian)-[~/STUDY/Manual/DSBDA/EXP12]
└─$ scalac Calculator.scala && scala Calculator
```

Choose operation to perform (+,-,*,/,%): *

Enter first number: 12

Enter second number: 5

Result: 60

String.scala

```
object ExampleString {
  def main(args: Array[String]) {
    val text : String = "You are reading SCALA programming language.";
    println("Value of text is: " + text);
  }
}

(vmsin0@martian)-[~/STUDY/Manual/DSBDA/EXP12]
└─$ scalac String.scala && scala ExampleString
Value of text is: You are reading SCALA programming language.
```